DIGITAL LOYALTY SaaS PLATFORM

Technical & Product Specification for Software Development

Purpose: Build an original, scalable digital loyalty platform for businesses in Algeria and eventually other markets. The product is functionally inspired by modern loyalty platforms such as Fidelix, but must use its own branding, UX, code and product identity.

Primary objective: Let a business create a loyalty program, let customers join through QR/NFC without downloading an app, let employees award stamps/points and redeem rewards, and let customers optionally save the loyalty card to Apple Wallet or Google Wallet.

Core principle

Simple for the business owner. Fast for the employee. Frictionless for the customer.

2. USER ROLES

Role	Responsibilities
Business Owner	Business profile, loyalty programs, rewards, customers, employees, analytics, locations and settings.
Manager	Operational management within assigned business/location; permissions configurable.
Employee	Scan/search customer, add stamps/points, redeem rewards. No billing or owner settings.
Customer	Join program, view digital card, progress, rewards and history; present QR code for identification.
Platform Admin	Internal administration: businesses, platform activity, support and account controls.

3. CORE END-TO-END FLOW

```
BUSINESS SIGNUP → BUSINESS CREATES LOYALTY PROGRAM → REWARD CREATED → SYSTEM GENERATES JOIN QR/NFC URL → CUSTOMER SCANS/TAPS → CUSTOMER JOINS → DIGITAL CARD CREATED → EMPLOYEE SCANS CUSTOMER → STAMP/POINT ADDED → TRANSACTION RECORDED → REWARD UNLOCKED → EMPLOYEE REDEEMS → ANALYTICS UPDATED.
```

5. LOYALTY PROGRAMS

Stamp model: Example: 10 visits/stamps = Free Coffee. Business sets required stamps and reward.

Points model: Example: 1 point per 100 DZD spent. Business defines earning rule and reward thresholds.

The data model must support both models from the beginning without requiring a future database redesign.

7. EMPLOYEE / COUNTER FLOW

Employee clicks **Scan Customer** → camera opens → customer QR scanned → membership verified → customer/program shown → employee chooses Add 1 Stamp / Add Points → confirmation → transaction created → balance/progress updated.

Prevent accidental duplicate submissions with server-side idempotency/duplicate protection and clear success feedback.

8. REWARDS & REDEMPTION

• Reward fields: name, description, image (optional), required stamps/points, expiration (optional), active/inactive.

• Customer becomes eligible when threshold is reached.

• Employee scans customer/reward and confirms redemption.

• Server verifies customer, business, program, reward eligibility and redemption status.

• Create immutable redemption record with customer, business, location, employee, reward and timestamp.

• Prevent double redemption.

9. CUSTOMER QR / NFC

Business join QR/NFC: contains only a public loyalty-program URL such as /join/{programId}. NFC is simply an easy physical way to open that URL.

Customer QR: use a secure random membership identifier/token. Never encode sensitive personal information directly in the QR.

The backend resolves the identifier to the correct membership and verifies authorization before allowing any action.

11. DATABASE / DATA MODEL

Core tables: users, businesses, business_locations, employees, loyalty_programs, customers, customer_memberships, rewards, transactions, reward_redemptions, customer_events.

Wallet tables later: wallet_passes and any required Apple/Google device/object mappings.

General fields: UUID primary keys, created_at and updated_at where appropriate; foreign keys and indexes for business_id, customer_id, loyalty_program_id, phone/email and timestamps.

Important: loyalty actions should create immutable transaction records rather than simply overwriting history. Balances may be maintained as derived/validated state, but the transaction ledger remains the audit trail.

12. SECURITY & MULTI-TENANCY

• Use Supabase Authentication and PostgreSQL.

• Use Row Level Security for every business-owned table.

• A business must never be able to query another business's customers, programs or transactions.

• Employees can only access permitted business/location data.

• Client-side authorization must never be trusted; enforce permissions server-side/database-side.

• Validate all inputs and ownership relationships.

• Use secure random IDs/tokens for public/customer QR identifiers.

• Protect against duplicate transactions and reward redemptions.

• Do not expose secrets in client-side code.

13. RECOMMENDED TECHNOLOGY

Frontend: React + TypeScript + Tailwind CSS, with a clean responsive component architecture.

Backend/data: Supabase + PostgreSQL + Supabase Auth + Row Level Security.

Hosting: Vercel-compatible deployment.

QR/NFC: QR generation + URLs; NFC tags simply point to the public join URL.

Keep the architecture modular so external services can be added later without rewriting the core loyalty system.

14. APPLE WALLET & GOOGLE WALLET

• Add real wallet integration, not a visual imitation.

• Apple Wallet: generate legitimate signed .pkpass files using Apple's PassKit format and Apple-issued signing credentials.

• Apple pass should contain business branding, customer name/ID, current stamps/points, reward information and a customer QR/barcode.

• Design for pass updates when loyalty balance changes; store pass serial/device relationships required by Apple's update mechanism.

• Google Wallet: use the official Google Wallet API with LoyaltyClass/LoyaltyObject concepts.

• Generate official Google Wallet save flow using server-side credentials.

• Never expose Apple private keys or Google service-account credentials to the frontend.

• Wallet credentials should be environment secrets; use placeholders in development.

• Do not claim production wallet integration is complete until real credentials and external platform configuration are present.

15. WALLET UPDATE FLOW

```
Employee adds stamp → transaction saved → membership balance changes → backend triggers wallet update → Apple pass/Google object reflects new balance. Reward unlocking and redemption should similarly update wallet-visible status where supported.
```

16. UI / UX DIRECTION

• Premium, modern, minimal SaaS appearance.

• Mobile-first customer experience.

• Responsive merchant dashboard.

• Strong typography, generous spacing, rounded cards, subtle shadows and restrained use of color.

• Avoid clutter and unnecessary animations.

• Customer card should feel like a digital membership card, not an admin dashboard.

• Business owner should understand the onboarding process without technical knowledge.

17. MVP SCOPE

• Authentication and roles; business onboarding; dashboard; stamp and points programs; rewards; customer registration; mobile digital card; business join QR/NFC URL; customer QR; employee scan/add stamp or points; reward redemption; customer and employee management; basic analytics; secure multi-tenant database.

Defer until core MVP works:

• Apple Wallet and Google Wallet production credentials/integration.

• WhatsApp/SMS/email automation; referral programs; Google review campaigns; subscription billing; advanced analytics/AI; custom domains/white-labeling; native mobile apps; offline mode.

18. DEVELOPMENT PHASES

Phase	Deliverables
1. Foundation	Project structure, Supabase, authentication, schema, RLS, roles, basic app shell.
2. Merchant	Business onboarding, dashboard, programs, rewards.
3. Customer	Join flow, membership, mobile digital card.
4. Operations	Customer QR, employee scanner, stamp/point transactions.
5. Rewards	Eligibility, redemption, redemption history.
6. Management	Customer management, employees, analytics.
7. Wallet	Apple Wallet + Google Wallet integration and update architecture.
8. Production	Security review, testing, monitoring, deployment and documentation.

19. BUSINESS MODEL (INITIAL CONCEPT)

Initial target market: Algeria. Suggested pricing is a business hypothesis and should be validated with real merchants.

Plan	Illustrative monthly price	Potential contents
Starter	1,500 DZD	Digital loyalty, QR/NFC join, stamps/points, rewards, basic dashboard.
Pro	3,000 DZD	Analytics, employees, automations and additional retention features.
Business	5,000–8,000 DZD	Multiple locations, advanced analytics, automation and priority support.

Physical NFC/QR plates can be sold separately as an acquisition/onboarding product. Pricing is to be validated.

20. PRODUCT POSITIONING

The goal is not merely a digital stamp card. The long-term product should become a customer-retention platform: loyalty + customer database + rewards + automated re-engagement + referrals + review campaigns + wallet presence.

The first commercial milestone is intentionally narrow: prove that a real business can create a program, customers can join quickly, employees can issue loyalty value in seconds, rewards can be redeemed safely, and the owner can see useful data.

21. ACCEPTANCE CRITERIA FOR MVP

• A new business can sign up and create its first loyalty program without developer intervention.

• A customer can join using a QR/NFC link on a phone in under approximately 30 seconds.

• A customer receives a persistent digital loyalty card.

• An employee can identify a customer and award loyalty value in a few seconds.

• Every loyalty action is recorded and auditable.

• A reward can only be redeemed when eligible and cannot be redeemed twice.

• Business data is isolated securely from other businesses.

• Dashboard numbers match the underlying transaction data.

• The application works on modern mobile and desktop browsers.

• The architecture is ready for Apple Wallet, Google Wallet and future integrations.

22. IMPORTANT DEVELOPMENT PRINCIPLES

• Do not build fake functionality or hardcoded production flows.

• Do not over-engineer the MVP.

• Build in phases and test each phase before moving to the next.

• Keep the database schema normalized and secure.

• Keep business logic on trusted server/database layers.

• Use reusable components and typed interfaces.

• Document environment variables and external credentials.

• Keep the code in the project owner's Git repository and keep third-party account ownership under the project owner.

23. HANDOFF NOTE TO THE SOFTWARE ENGINEER

This document describes the product direction and core requirements. The engineer should convert it into a technical implementation plan, confirm architecture decisions, identify external dependencies, estimate each phase, and flag any requirement that needs clarification before implementation. The first priority is a reliable, secure MVP—not a large collection of unfinished features.

END OF SPECIFICATION